

Missile Trajectory Optimization Using a Modified Ant Colony Algorithm

Zachary J. Kiyak
Department of Aerospace Engineering
Auburn University
Auburn, AL 36849
256-431-4075
zjk0002@auburn.edu

Roy J. Hartfield
Department of Aerospace Engineering
Auburn University
Auburn, AL 36849
334-444-8523
rjh@eng.auburn.edu

Timothy W. Ledlow II
Department of Aerospace Engineering
Auburn University
Auburn, AL 36849
256-345-9476
twl0001@auburn.edu

Abstract—A modified ant colony optimization (ACO) algorithm is applied to a single-stage solid missile design problem involving six degrees of freedom (6-DOF) flight trajectory modeling. A local search procedure is also integrated with the algorithm, adding a search intensification ability that compliments the ability of ACO to thoroughly explore a solution space. The goal of this work is to evaluate the effectiveness of the ant colony optimization scheme by comparing its solution output quality with those of other, well-known optimization methods. Performance is based on solution “fitness”, or how closely each solution matches a specific set of performance objectives, as well as the number of calls to the objective function that are required in order to reach that solution. Additionally, an important performance criterion is to determine each algorithm’s capabilities of finding, not only a single quality solution to a design problem, but also a diverse set of additional, near-optimal solutions. The results of this study demonstrate that the modified ACO algorithm is a viable candidate as a high-performance optimization method, while the added local search method may or may not be a worthwhile addition for the attempted optimization problem.

TABLE OF CONTENTS

1	INTRODUCTION.....	1
2	ANT COLONY OPTIMIZATION.....	1
3	ALGORITHM MODIFICATIONS AND DEVELOPMENT.....	3
4	PROBLEM DESCRIPTION	4
5	RESULTS	5
6	SUMMARY	6
	REFERENCES	7
	BIOGRAPHY	7

1. INTRODUCTION

As computational technology has continued to advance, the integration of optimization methods with modeling and simulation has become an increasingly valuable tool for engineering design purposes. This union has been particularly useful in situations where it is especially difficult or impossible to develop an analytical solution to a given problem.

Numerous successful optimization strategies have recently

been developed and refined for this purpose. Amongst the most popular of these strategies are Genetic Algorithms (GAs), based on the theory of evolution, and Particle Swarm Optimization (PSO), [1] which is inspired by the principles of natural flocking behavior and social communication.

Other less well-known methods have also been used, along with various hybrids that attempt to combine strengths of different optimization schemes. However, aerospace design implementations of non-hybrid, ant colony methods have rarely been attempted.

Motivation

One important class of problems within aerospace engineering, is that of 6-DOF missile design and trajectory matching. These are often highly complex problems well-suited for heuristic optimization since preliminary level modeling is very effective in reducing development costs. Also, trajectory matching is very useful for a variety of tactical and reverse engineering purposes. Such problems have been attempted in several previous studies as listed in chapter 4, mostly using GA variants. In an attempt to discover an alternative heuristic, the ACO paradigm was chosen to be tested on a set of trajectory matching problems, because it was thought to be a promising candidate for not only finding a single optimum solution, but also multiple solutions to satisfy a given objective. This reasoning comes from a previous study involving the optimization of star-grain solid rocket motors to match a pre-specified actual or geometrical chamber pressure versus time profile (see figure 1), where it was found that ACO performed admirably [2]. The decision to use the Simplex local search method for this problem also comes from this study, where it was found to outperform other direct search procedures, including Conjugate Directions, Compass Search, and Pattern Search procedures. It was hoped that the Simplex Method would enjoy similar advantages for the 6-DOF problems, although the solution space is much different and is of higher complexity.

2. ANT COLONY OPTIMIZATION

Natural Inspiration

ACO algorithms are inspired by the foraging behaviors of some species of ants found in nature where it has been observed that a colony of ants is able to select paths of minimal

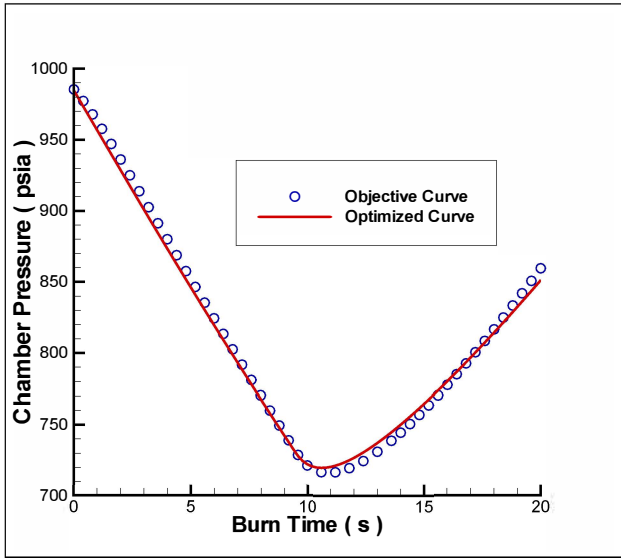


Figure 1: Solid Rocket Motor Optimization Example

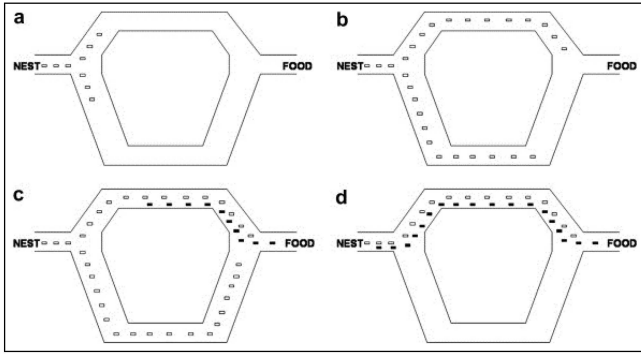


Figure 2: Double Bridge Experiment [5].

length to sources of food [3, 4]. The mechanism by which this is accomplished is the creation of bio-chemical trails that serve as a method of communication between individual members of the colony. This chemical, which is detectable by other members of a colony is known as trail pheromone.

Ants initially explore the area around their home nest in a random fashion. Upon encountering a food source, an ant will return to the nest, emitting a layer of pheromone to mark the path it has traveled. Other members of the colony are drawn toward this path as they probabilistically choose their foraging route based on the amount of pheromone present. These ants in turn emit their own pheromone, reinforcing good paths for others to follow. Eventually, the ants in the colony converge onto a single trail until the food source is exhausted, at which point the pheromone trail slowly evaporates. The evaporation effect allows the ants to explore new regions until a new food source is discovered. This explains how ants are able to recruit other colony members to one particular path, but for a large colony multiple paths to the same food source are often discovered. How then do the ants determine the best path? As a simple illustration, consider Figure 2. If a number of ants begin a journey from a nest and come to the first decision point, it was discovered experimentally by Goss [3], that stochastically half of them will travel along the high path while half will choose the lower path. Assuming the ants are moving at the same rate of speed, those that choose the

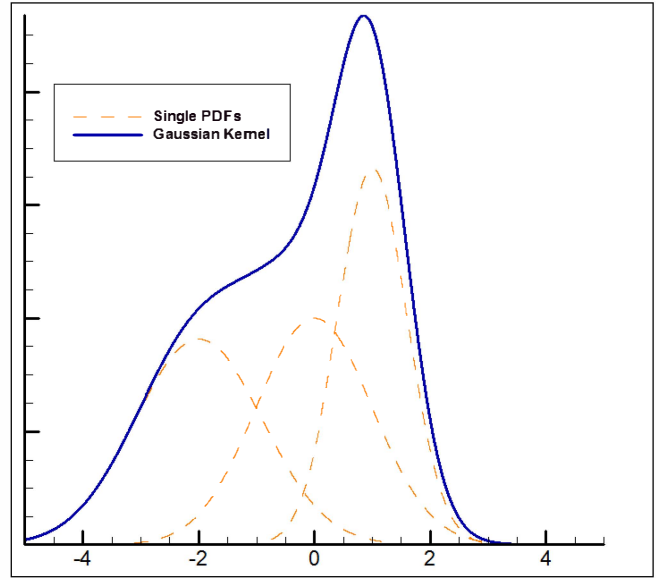


Figure 3: Gaussian Kernel

short path are the first to reach the food source and begin their return journey, thereby establishing a pheromone link more quickly than the ants that chose the longer route. In addition, pheromone that is added to the shorter path evaporates more slowly than that on the longer path. As ants continue to leave the nest they are increasingly drawn toward the shorter path. This selective bias continues to be reinforced as more ants complete the circuit until eventually almost all of the ants converge upon the optimal route.

Artificial Algorithm

The natural ant model became the inspiration for the ACO algorithm, initially proposed by Dorigo in 1992 [6] where a population of artificial ants can be used to iteratively create and improve solutions to a given optimization problem. ACO adopts the foraging principles from real ant colony behavior while also implementing several improvements including a tunable pheromone evaporation rate and an augmented ability for ants to calculate the quality of a particular solution and distribute differing levels of pheromone accordingly. After its introduction, many highly effective variants of the ACO algorithm were developed and implemented to solve a wide range of difficult combinatorial optimization problems including the well known quadratic assignment and traveling salesman problems where ACO has been proven to be very successful.

Despite its success in the combinatorial realm, the algorithm was difficult to apply to problems of a continuous nature, as the ant colony paradigm is not intuitively applicable to problems of this type. Many early attempts were made, however. The algorithms developed from many of these attempts borrowed several features of the original ACO model, but they did not follow it closely. In order to address this issue, Socha and Dorigo developed an algorithm in 2005 they named ACO-R that more closely adopted the spirit of the original method and could operate in continuous space natively [7]. The fundamental concept of ACO-R is that the discrete, probability distributions used in ACO are replaced by probability density functions (PDFs) in the solution construction phase, and the pheromone distribution is accomplished by a large solution archive of weighted PDFs that combine to form a

Gaussian kernel as shown in Figure 3. Illustrated in figure 4, the ACO-R algorithm methodology is as follows:

First, the solution archive is filled with randomly generated solutions. These solutions are then ranked and given weights based on their quality.

$$w_j = \frac{1}{qk\sqrt{2\pi}} \exp \frac{-(rank(j) - 1)^2}{k - 1} \quad (1)$$

where j is a solutions index in the archive, k is the total number of solutions, and q is a parameter of the algorithm that determines how the weights are distributed. Lower values result in more elitist weight distributions, whereas higher values create more uniform distributions. These weight assignments are analogous to differing pheromone amounts and bias the selection of higher quality solutions within the search space. Each time a new “ant” constructs a solution, a new PDF is selected for each dimension from among the solution archive with probability proportional to its weight. In this way, all of the solutions are given a chance to contribute, producing a diverse and robust exploration strategy. After a given PDF has been chosen for each component, a new solution is taken from a normal distribution oriented around its center. The standard deviation of this distribution is calculated using equation 2.

$$\sigma_j^i = \zeta \sum_{r=1}^k \frac{abs(s_i^r - s_i^j)}{k - 1} \quad (2)$$

This equation sets the standard deviation of each variable i as dependent on the average distance between the corresponding variable components of all the other k solutions in the archive. In this way, the characteristics of each Gaussian PDF selected by the ants are affected by the positions of all the other ants in the archive. Pheromone evaporation rate is governed by the parameter ζ where a smaller value encourages fast convergence and a larger value encourages a more thorough exploration of the search domain.

Once m new solutions have been generated, they replace the m worst solutions in the current archive which is then re-sorted. This process functions in the same way as pheromone update by removing the impact of the worst solutions in the current pheromone model. It also ensures that new solutions will have at least some chance of being selected in the following iteration, regardless of their fitness, thus encouraging new exploration.

An optional final step of the original ACO-R algorithm is to correlate variables using an orthogonal gram-schmidt process. For problems involving many variables, however, the computational effort required to implement this step has been found to be inefficient [8].

3. ALGORITHM MODIFICATIONS AND DEVELOPMENT

The ACO-R algorithm of Socha and Dorigo (sans variable correlation) was adopted as the basis of the algorithm while several modifications were made in order to improve its ability to converge quickly and efficiently to a quality solution. At the same time, it was important to ensure that the algorithm was able to avoid stagnation and continuously explore the solution space for multiple potential solutions.

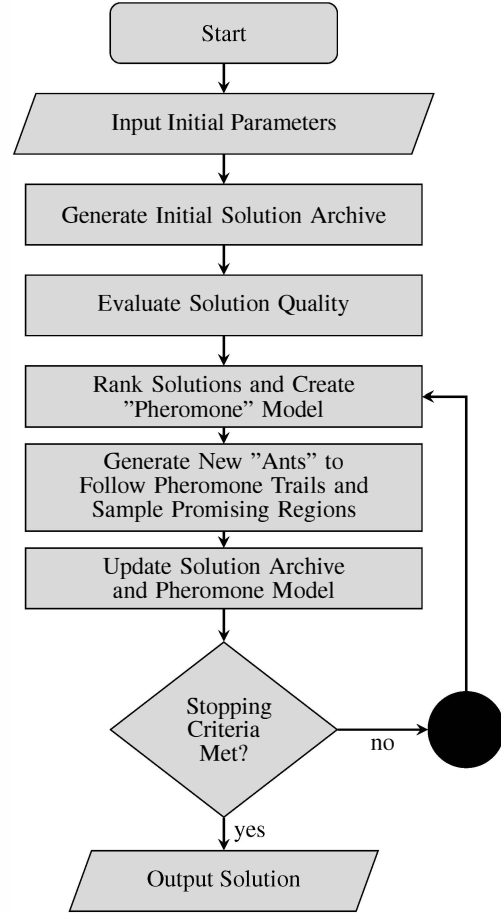


Figure 4: ACO Algorithm Flow

Elitism

The first modification was that a higher emphasis was placed around searching the best-so-far solution in the archive in order to achieve quicker convergence. An extra tuning parameter q_{mod} was created that controlled the probability that a newly generated “ant” sampled only the area around the best solution. A similar approach was taken by ref [8]. The range for this parameter was set to a decimal number between 0 (best solution never used) and 1 (best solution always used). The reason for this modification was that after preliminary testing of the algorithm it was felt that the original q weighting parameter was not able to give the amount of control wanted over convergence behavior. When elitism was attempted by setting the weighting parameter to a very high value, the algorithm tended to converge very quickly, but at the expense of greatly reduced exploration ability. When the weighting parameter was set low, the algorithm tended to become unguided and inefficient. By guaranteeing that ants searched a certain percent of the time around the current best solution, while at the same time allowing more of the solutions at the bottom end of the archive to contribute, both exploration and intensification were encouraged.

Local Search

The second modification was the addition of the Nelder-Mead Simplex Method as an integrated local search procedure. This was done in an effort to intensify the search around a good solution after the solution archive in each ACO cycle had converged.

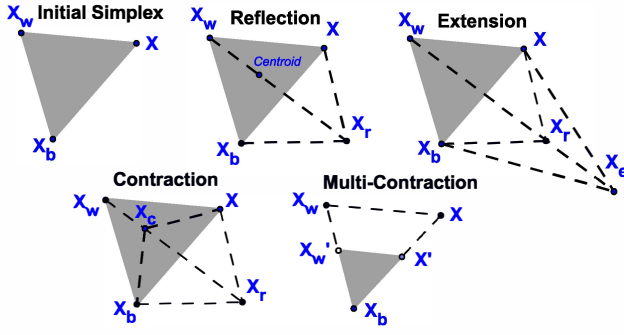


Figure 5: Simplex Morphology [10]

Simplex Method—First developed in 1965 [9], the Nelder-Mead Simplex Method is a well-known derivative-free, non-linear optimization technique. The simplex algorithm operates by creating a system of $N+1$ vertices for an N -dimensional problem which then undergoes a series of reflections and expansions in order to iteratively crawl its way towards a local optimal or near-optimal solution. For example, in two dimensions, the simplex takes the form of a triangle and in three dimensions, a tetrahedron. When consecutive iterations of simplex evolution are animated, it appears as though the simplex structure morphs its way toward a solution, earning it the often-used alternate name of the Amoeba Method. Figure 5 illustrates the various ways the simplex structure can evolve within two-dimensional space.

The rules governing simplex behavior can be described as follows:

- 1) Construct an initial simplex structure around the base point x_b .
- 2) The point with the worst initial fitness x_w is reflected through the centroid of the simplex.
- 3) If the reflected point x_r is a new best solution, expand the simplex and reflect further x_e .
- 4) If the new point is not a new best solution, but is still a good solution, start at the top and reflect again.
- 5) If the new point is a bad solution, contract the simplex x_c and return to step two.
- 6) If successive contractions fail to find an improvement, contract each point x' within the simplex structure.

The simplex method is much more aggressive than many other well-known direct search techniques such as the compass and pattern search procedures. This allows it to converge much more quickly over a well-behaved solution space, although a downside of this characteristic is that, at times, the simplex may be more easily confused by complex problems. In an attempt to combat this drawback, a restart mechanism was included within the simplex subroutine which allowed the algorithm to reset mid-run if needed.

Multiple Optima

The final modification to the algorithm, was that it was coded to perform using a series of fast-converging mini-cycles.

This was done in order to combat stagnation and allow the algorithm to explore a wider range of solutions within the design space. An optional switch was also encoded into the algorithm which served to prevent the algorithm from re-exploring combinations of variables that were highly similar to those found in previous cycles.

4. PROBLEM DESCRIPTION

Objectives

The first design objective for the algorithm was chosen to be the optimization of a single-stage missile to match a pre-determined trajectory as closely as possible. The second, more complicated objective was to match not only one trajectory, but three simultaneously.

Individual missile designs, using the available flight simulation programs, are composed of 35 essential design parameters, of which 17 determine missile geometry, 7 govern propellant properties, and 11 compose the autopilot control system. Each parameter can be varied by an optimizer to change the missiles flight characteristics in order to meet a set of pre-specified performance objectives designated by the user. This creates a very complex design space, well-suited for challenging an optimizer's capabilities, especially in the multiple trajectory case.

Single Trajectory Problem

In order to match the 164 km, single trajectory, it was important to set up an appropriate objective function capable of effectively evaluating how closely an individual missile design actually matched the trajectory model. For this problem, the overall objective function was composed of four separate flight objectives: (1) Time of flight (2) Time until burnout (3) Apogee match and (4) Range of flight. Each objective was given a weight in proportion to its estimated significance, and the final evaluation of the objective function produced a quality or "fitness" given by equation 3, where the denominator of each term reflects the assigned weighting, and a lower value for fitness corresponds to a better solution.

$$Fitness = \frac{RangeError}{10} + \frac{ApogeeError}{10} + \frac{TOFError}{1} + \frac{TBOError}{1} \quad (3)$$

Triple Trajectory Problem

The second problem attempted was to match three separate trajectories, one of which was the 164 km, single trajectory used in the first problem. The additional trajectories had ranges of 117km and 221km. All three are shown in figure 6. It is sometimes useful to design a missile that can match multiple trajectories. This feat is much more difficult than matching a single trajectory alone, as the design space becomes more complex and there are more opportunities for an optimizer to become confused. The fitness of each missile design was calculated using equation 3 as before, with the final fitness equating to the sum total obtained from each of the three trajectory matches.

$$Fitness = Fitness_{117} + Fitness_{164} + Fitness_{221} \quad (4)$$

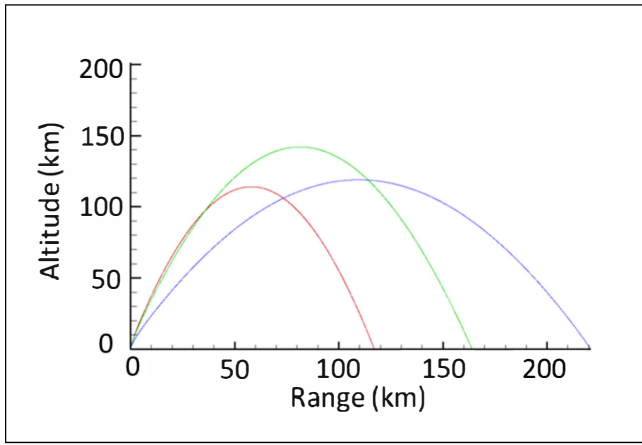


Figure 6: Three Missile Trajectories

6-DOF Missile Code

The primary tool used for this study was a missile design program developed by Burkhalter, Jenkins, Hartfield, Anderson, and Sanders [11] which is capable of creating and flying preliminary level engineering models of missiles powered by a single-stage solid propellant motor. This code has proven to be a reliable tool for aerospace design applications and has been utilized in many previous optimization studies [12–14].

Before a viable 6-DOF trajectory can be simulated by the missile program, several processes must first occur. For each missile design:

- 1) The propulsion system is modeled using grain geometries specified by the input to develop the thrust profile for the motor.
- 2) Mass properties are calculated and used to determine the center of gravity and moments of inertia for all components included in the system
- 3) Aerodynamic properties are calculated using the non-linear, fast predictive code AERODSN [15]

The resulting propulsion, mass, and aerodynamic characteristics are then integrated with the equations of motion to determine the flight profile for the missile.

Validation

A binary genetic algorithm developed by Anderson [16] known as IMPROVE (Implicit Multi-objective PaRameter Optimization Via Evolution) is used for comparison and validation for the modified ACO algorithm. This algorithm has been used multiple times in previous optimization studies and has been shown to be an effective and robust tool for aerospace design optimization [17–19].

In order to accurately compare results between ACO and IMPROVE, the local search ability in the ACO algorithm was turned off. The GA algorithm parameters were not varied for this problem as the default settings had been successfully used on previous, similar design problems. Similarly, the ACO tuning parameters were not adjusted from their values used in the solid rocket motor problem motivating this study. In a further attempt to avoid an unfair comparison, the Binary GA was modified to work with multiple, fast-converging cycles in a similar manner to ACO. Two different population

sizes composed of 50 and 200 members were also attempted. Only the results from the best of the Binary GAs was used in comparison with ACO.

For every run, each optimizer was allowed to perform 40,000 function calls to the objective function.

5. RESULTS

Single Trajectory Results

As illustrated in figure 7, the ACO algorithm was able to outperform the Binary GA, converging much more rapidly and ultimately finding a better solution with a fitness of 8.20 compared with 10.74 for the GA. Additionally, many unique alternate solutions were found during the ACO run with very good fitness values as shown in Table 1, demonstrating its ability to find multiple, near-optimal solutions to the same problem. The final missile design obtained by each optimizer is shown in figure 8.

Multiple Trajectory Results

The ACO algorithm performed very well, as illustrated in figure 10, achieving a missile with a final fitness value of 46.31 compared with the 229.86 found by the best GA. In addition to the best solution, depicted in figure 9, the ACO optimizer was able to find four different missile designs during the course of its run with fitness values below 100 as shown in Table 2.

Local Search

The addition of the integrated local search provided some improvement to the solutions as shown in Tables 3 and 4, but the improvement came at the cost of adding 10,000 function calls to the program. So although the addition resulted in a better final solution, it came at a cost that could have been spent on further exploration by the main algorithm. The results may indicate that the ACO algorithm is able to get quite close to a local optimum value without the aid of a local search, or it may also indicate that the Simplex Method is not suited for the problem's design space, especially for the multiple trajectory problem where there was a lower amount of improvement.

Table 1: Final Optimization Results

Optimizer	Single Trajectory Best	3 Trajectory Best
Binary GA	10.74	229.86
ACO	8.20	46.31
ACO + LS	7.61	40.40

Table 2: Multiple Near-Optimal Solutions Found by ACO

Solution No.	Single Trajectory	Three Trajectories
1	8.55	61.35
2	8.95	65.37
3	9.27	83.04
4	9.32	93.22

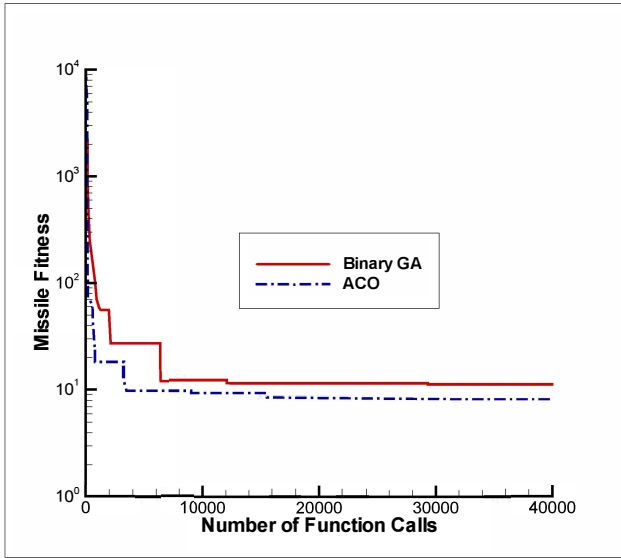
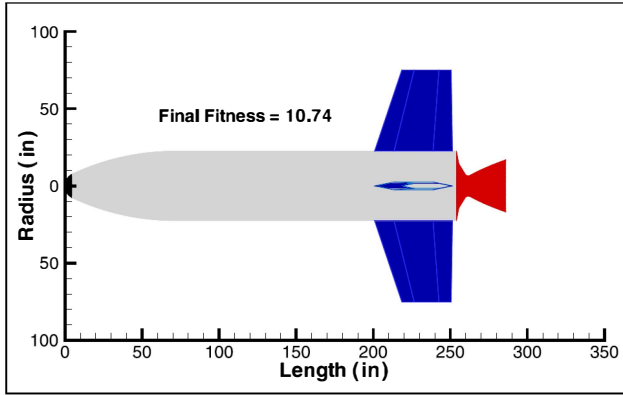
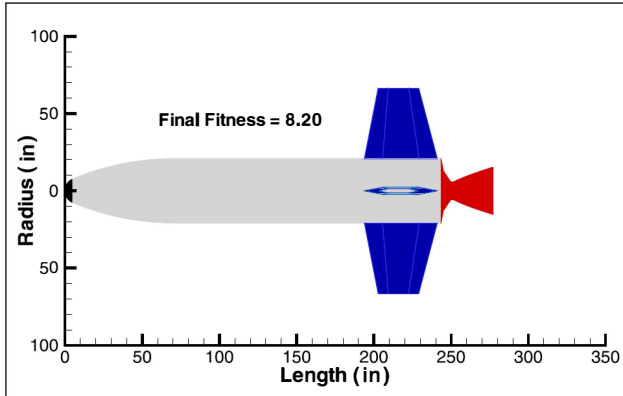


Figure 7: Convergence History for Single Trajectory Case



(a) GA Best Solution

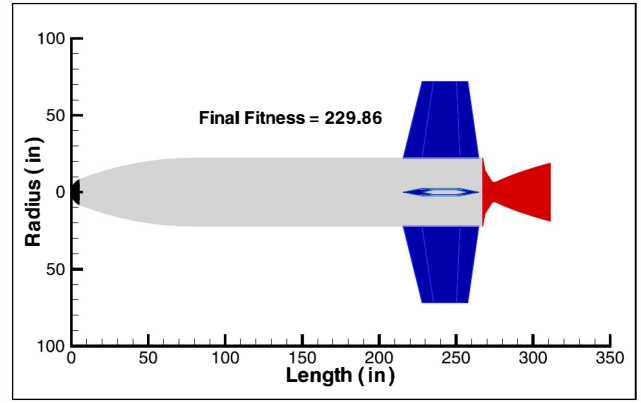


(b) ACO Best Solution

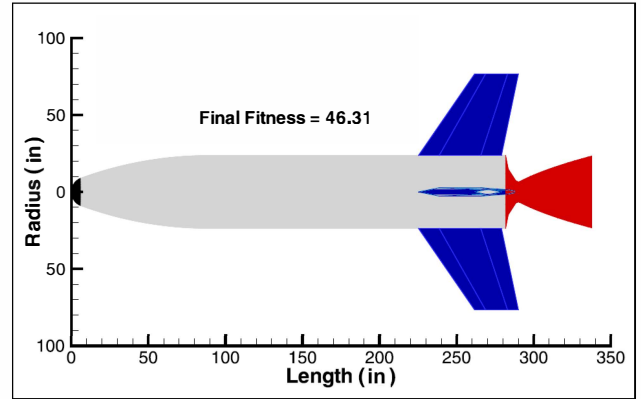
Figure 8: Optimized Missile Designs - Single Trajectory

6. SUMMARY

Preliminary testing of the modified ACO algorithm indicates that it is a capable alternative method that should merit consideration as a high-functioning optimizer. It also showed great promise as a tool for finding multiple solutions within a single run. The local search feature was able to improve



(a) GA Best Solution



(b) ACO Best Solution

Figure 9: Optimized Missile Designs - Three Trajectories

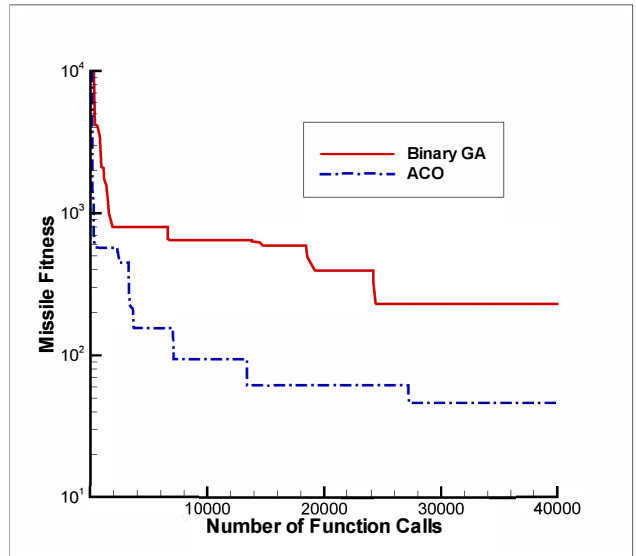


Figure 10: Three-Trajectory Convergence History

some solutions, but the results indicate that it may not be worth the cost in efficiency needed to facilitate its operation within the algorithm. It is suggested that for this and other problems of similar complexity, the local search be applied only after the main algorithm has been run. It can then be used supplementally, to improve the very best of the solutions discovered previously.

Table 3: Local Search Improvements - 164km Trajectory

Initial Step Size (% range)	Average Improvement (%)
0.5	12.3316
1.0	11.1657
2.0	11.5382

Table 4: Local Search Improvements - Three Trajectories

Initial Step Size (% range)	Average Improvement (%)
0.5	4.6837
1.0	4.8881
2.0	4.6092

Future work will involve performing multiple runs in order to tune the algorithm as well as demonstrate that quality results are repeatable. Applications to other design problems will also be attempted. Finally, comparisons to state-of-the-art algorithms in current use will be performed to determine its place amongst the modern suite of optimization methods available for aerospace design applications.

REFERENCES

- [1] Kennedy, J., and Eberhart, R. "Particle Swarm Optimization". Proceedings of IEEE International Conference on Neural Networks IV. pp. 1942-1948, 1995.
- [2] Kiyak, Zachary J., Hartfield, Roy J., "Solid Rocket Motor Design Using a Modified Ant Colony Optimization Metaheuristic with Local Search Capability", AIAA 2013-3788 presented at the Forty Ninth AIAA/ASME/SAE/ASEE Joint Propulsion Conference, San Jose, CA, July 14-17, 2013.
- [3] Goss, S., Aron, S., Deneubourg, J.L., and Pasteels, J.M., Self-organized Shortcuts in the Argentine Ant, *Naturwissenschaften*, Vol. 76, pp. 579-581, 1989.
- [4] Hoelldobler, B., and Wilson, E. *The Ants*, Belknap Press, 1990.
- [5] Cordon, O., Herrera, F., and Stutzle, T., A review on the ant colony optimization metaheuristic: Basis, models and new trends., *Mathware and Soft Computing*, Vol. 9, pp. 141-175, 2002.
- [6] Dorigo, M., *Optimization, Learning, and Natural Algorithms*, PhD thesis, Politecnico di Milano, Italie, 1992.
- [7] Socha, K., Dorigo, M., *Ant Colony Optimization for Continuous Domains*, *European Journal of Operations Research*, Vol. 185, 2008, pp. 1155-1173.
- [8] Liao, Tainjun, Montes de Oca, Marco, A., Aydin, Dogan, Stutzle, Thomas., and Dorigo, Marco., "An incremental ant colony algorithm with local search for continuous optimization.", *Proceedings of the 13th annual conference on Genetic and evolutionary computation*, pp. 125-132, April, 2011.
- [9] Nelder, J.A., and Mead, R., A Simplex Method for Function Minimization, *Computer Journal*, Vol. 7, No. 4, pp. 308-313, 1965.
- [10] C. H. Tse, M. Tang, P. P. Shum, and R. F. Wu, Nelder-mead simplex method for modeling of cascaded continuous-wave multiple-stokes raman fiber lasers, *Optical Engineering*, vol. 49, no. 9, 2010.
- [11] J.E. Burkhalter, R.M. Jenkins, and R.J. Hartfield, M. B. Anderson, G.A. Sanders, *Missile Systems Design Optimization Using Genetic Algorithms*, AIAA Paper 2002-5173, Classified Missile Systems Conference, Monterey, CA, November, 2002
- [12] Hartfield, Roy J., Jenkins, Rhonald M., Burkhalter, John E., *Ramjet Powered Missile Design Using a Genetic Algorithm*, AIAA 2004-0451, presented at the forty-second AIAA Aerospace Sciences Meeting, Reno NV, January 5-8, 2004.
- [13] Jenkins, Rhonald M., Hartfield, Roy J., and Burkhalter, John E., *Optimizing a Solid Rocket Motor Boosted Ramjet Powered Missile Using a Genetic Algorithm*, AIAA 2005-3507 presented at the Forty First AIAA/ASME/SAE/ASEE Joint Propulsion Conference, Tucson, AZ, July 10-13, 2005.
- [14] Dyer, John D., Hartfield, Roy J., Dozier, Gerry V., and Burkhalter, John E., *Aerospace Design Optimization Using a Steady State Real-Coded Genetic Algorithm*, AIAA Paper 2008-5921, 12th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference, September 2008.
- [15] Sanders, G.A. and Washington, W.D., *Computer Program for Estimating Stability Derivatives of Missile Configurations - Users Manual*, U.S. Army Missile Command, August 1982.
- [16] Anderson, M. *IMPROVE (Implicit Multi-objective Parameter Optimization Via Evolution)*, An Optimization Software Package Based on Genetic Algorithms, Department of Aerospace Engineering, Auburn University, 1996.
- [17] Bayley, D., Hartfield, R., Burkhalter, J., and Jenkins, R., *Design Optimization of a Space Launch Vehicle Using a Genetic Algorithm*, AIAA Paper 2007-1863, presented at 3rd AIAA Multidisciplinary Design Optimization Specialist Conference, Honolulu, Hawaii, April 23-26, 2007
- [18] Metts, J., Hartfield, R., Burkhalter, J., and Jenkins, R., *Reverse Engineering of Solid Rocket Missiles with a Genetic Algorithm*, AIAA Paper 2007-0363, presented at the Forty-Fifth Aerospace Sciences Meeting and Exhibit, Reno, NV, Jan 8-11, 2007.
- [19] Riddle, D., Hartfield, R., Burkhalter, J., and Jenkins, R., *Design of Liquid Rocket Powered Missile Systems Using a Genetic Algorithm*, AIAA Paper 2007-0362, presented at the Forty-Fifth Aerospace Sciences Meeting and Exhibit, Reno, NV, Jan 8-11, 2007.

BIOGRAPHY

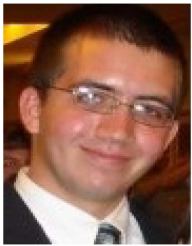


Zachary J. Kiyak received a B.S. in Physics from the University of North Alabama in 2008 and is currently pursuing a M.S. in Aerospace Engineering at Auburn University. Under the direction of Dr. Roy J. Hartfield, his primary research interest is the application and development of optimization algorithms for solid-rocket-motor design and 6-DOF missile analysis.



Roy J. Hartfield is a Walt and Virginia Woltosz Professor of Aerospace Engineering in the Department of Aerospace Engineering at Auburn University. He received the Ph.D. degree in Mechanical and Aerospace Engineering from the University of Virginia in 1991. Dr. Hartfield joined the Aerospace Engineering department at Auburn immediately after completing his Ph.D.

Dr. Hartfield's research areas have included engine development, aerodynamic measurement technology, high speed air breathing propulsion, rocket propulsion, vehicle aerodynamics, missile system modeling, optimization of aerospace systems, and risk assessment for unmanned aerial vehicles. Dr. Hartfield is an associate fellow of the American Institute of Aeronautics and Astronautics (AIAA) and chaired the 2006 AIAA Applied Aerodynamics Conference. Dr. Hartfield currently serves as a member of the AIAA High Speed Air Breathing Propulsion Technical Committee and frequently teaches short courses in the area of propulsion. Dr. Hartfield has published 25 referred journal articles and approximately 75 peer reviewed conference papers.



Timothy W. Ledlow II received the Bachelor of Aerospace Engineering degree from Auburn University in Auburn, Alabama, in 2012. He is currently pursuing the M.S. degree in aerospace engineering at Auburn University under Dr. Roy Hartfield. His research interests include the development and application of optimization methods to complex engineering problems such as missile system design and grid fin optimization.